

第10章 类型标注与元编程

TypeVar 与 Generic · Protocol · 描述符 · 元类与 __init_subclass__

本章目标:讲透 Python 类型标注的进阶能力与"类的类"元编程:TypeVar/Generic 泛型、Protocol 结构化协议、描述符协议、元类与 __init_subclass__。学完你能写出既保持 Python 灵活、又能被 mypy 静态检查护航的生产代码,并理解装饰器、property 等语法糖背后的机制。

直觉起点:C 的类型在编译期决定一切;Python 的类型标注是"写给检查器与人看的承诺",运行期基本不检查。泛型之于 Python,类似 C 的宏之于类型——但更安全。元编程则是"创建类的代码"——class 语句本身就是一个可拦截的构造过程,Python 把 C 的编译期魔法搬到了运行期。

前置要求:第6章面向对象(类与继承)、第4章函数(标注语法)、第9章异常。

本章路线:第一阶段(1.5 小时)读 §10.1–§10.3,掌握泛型与 Protocol;第二阶段(1.5 小时)读 §10.4–§10.6,理解描述符与类创建钩子;第三阶段(2 小时)读 §10.7–§10.8,完成章末实验。

本章知识导图

- └─ 10.1 标注语法速览 —— X | Y、内置泛型、3.8 兼容
- └─ 10.2 TypeVar 与 Generic —— 泛型函数与容器
- └─ 10.3 Protocol —— 鸭子类型的静态化
- └─ 10.4 描述符 —— __get__/__set__ 与 property 真相
- └─ 10.5 元类 —— class 语句背后的构造过程
- └─ 10.6 __init_subclass__ —— 更亲民的子类钩子
- └─ 10.7 dataclass —— 数据类的元编程红利
- └─ 10.8 生产实践 —— 元编程的边界与检查工具

§10.1 标注语法速览:X | Y、内置泛型

标注只对检查器有意义,运行期几乎不校验——这是理解本章的前提。语法上记住四个点:内置容器可直接标注 (list[int]、dict[str, int]);可空用 X | None (3.10+)或 Optional[X];联合用 A | B (3.10+)或 Union[A, B];泛型参数给容器:

```
from typing import Optional, Union

# 3.10+ 新写法
def lookup(name: str) -> dict | None:
    ...

# 3.8 兼容写法:同一含义
def lookup_old(name: str) -> Optional[dict]:
    ...

# 3.8 里内置类型不能直接下标,要用 typing 版本
from typing import List, Dict, Tuple
def stats(scores: List[int]) -> Tuple[int, float]:
    ...
```

3.8 兼容注意: `list[int]` 这类内置泛型语法从 3.9 才有;3.8 必须写 `List[int]` 。若项目仍在支持 3.8,统一用 `typing` 版别称;项目已是 3.10+,全用新语法,代码更短。除了语法,还要记住 `->` 后的返回值标注里写 `"Point"` 字符串(前向引用)可以避免定义顺序问题,或直接用 `from __future__ import annotations` 延迟求值。

标注写错的代价不是崩溃而是误导:错误类型被检查器传给下游,错误像滚雪球。所以标注的纪律是"从入口到出口完整"——函数、类字段、返回值都要有,半标注比不标注更危险,因为它给了假的信心。

§10.2 TypeVar 与 Generic:泛型函数与容器

TypeVar 声明一个"类型变量",让一个函数或类对多种类型保持类型安全——同一参数位置的类型关系被检查器追踪,不会像 C 的 `void*` 那样失去类型信息:

```
from typing import TypeVar, Generic

T = TypeVar("T")                # 任意类型
U = TypeVar("U", int, float)    # 只允许数值类型

def first(items: list[T]) -> T:
    """泛型函数:返回类型与元素类型一致"""
    return items[0]

def add_all(a: U, b: U) -> U:
    """类型变量同一位置要求同类型"""
    return a + b

class Stack(Generic[T]):        # 泛型类:Stack[str]、Stack[int]
    """后进先出栈,元素类型在实例化时确定"""

    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

s = Stack[int]()
s.push(1)
n: int = s.pop()                # 检查器知道 pop 返回 int
```

C(void*/宏)	Python(泛型标注)
void* 传数据,类型信息丢失	T 被检查器全程追踪,返回类型精确
宏展开错误在编译期报,信息晦涩	类型错误在 mypy 运行时报,信息直白
强转(cast)全靠人肉保证	检查器验证类型关系,不一致即报错
运行期零开销	标注运行期零开销(不检查,但也不约束)

TypeVar 还有两个变体: `TypeVar("T", bound=Base)` 限制 T 必须是某基类或其子类; `TypeVar("T", covariant=True)` 用于"只读容器"的协变。日常 90% 用最朴素的 `TypeVar("T")` 就够。

泛型标注要避免过度设计:一个函数用三个 TypeVar 多半是设计出了问题。经验法则:类型变量出现在两个以上位置才有价值;如果 T 只出现一次,直接用具体类型或 Any 更诚实。

§10.3 Protocol:鸭子类型的静态化

第6章说"长得像就用",运行时这么写没问题,但检查器看不到"长得像"的契约。**Protocol** 把鸭子类型写成可检查的结构化协议——只要类型有这些方法,就被认定为协议实现者,无需继承:

```
from typing import Protocol

class Payable(Protocol):
    """结构化协议:有 pay() 方法即满足,无需继承"""

    def pay(self) -> float: ...

class Manager:                                # 完全不继承 Payable
    def pay(self) -> float:
        return 20000.0

def total_pay(staff: list[Payable]) -> float:
    return sum(p.pay() for p in staff)

print(total_pay([Manager()]))                # mypy 通过,运行正常
```

误区:拿 Protocol 当运行期检查。 Protocol 是给 mypy 看的契约, `isinstance(obj, Payable)` 默认直接报错——除非加 `@runtime_checkable` 装饰器,且只能做"方法存在性"检查。生产姿势:类型检查靠 Protocol,运行期防御靠 EAFP(直接调用 + 捕获 `AttributeError`),两层各司其职。

Protocol 的威力在大型项目:接口定义在库层,调用方与实现方互不认识也能类型联动;改协议时检查器立刻列出所有违约者。这比 C 的虚函数接口更松、比鸭子类型更严,是"Pythonic 的接口"。注意 Protocol 成员的 `...` 占位写法,以及协议方法不要写实现。

§10.4 描述符: `__get__` / `__set__` 与 property 的真相

描述符是实现了 `__get__` / `__set__` 的对象,被赋值给类属性时,对实例属性的访问会转交给它。property、classmethod、staticmethod 全是描述符——理解了描述符,就理解了这些语法糖的底层:

```

class Positive:
    """字段描述符:拒绝负值,自动校验"""

    def __init__(self, name: str) -> None:
        self.name = name          # 存储用的属性名

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self           # 类上访问时返回描述符本身
        return obj.__dict__[self.name]

    def __set__(self, obj, value) -> None:
        if value < 0:
            raise ValueError(f"{self.name} 不能为负")
        obj.__dict__[self.name] = value

class Account:
    balance = Positive("balance")  # 描述符实例作为类属性

    def __init__(self, balance: float) -> None:
        self.balance = balance    # 触发 __set__ 校验

a = Account(1000)
a.balance = 500                  # 正常
a.balance = -5                   # ValueError: balance 不能为负

```

误区:描述符的存储属性与描述符名冲突。 `__set__` 里如果写成 `obj.balance = value`,会再次触发 `__set__` 无限递归。正确姿势:把值存进实例的 `__dict__`,key 用描述符自己的名字(如 `self.name="balance"`),这是描述符的标准存储模式。另一个坑:描述符只在"类属性"上生效,塞进实例属性就完全失灵。

看完这个例子就明白 `@property` 是什么:Python 内置的 `property` 描述符,把 `fget / fset` 两个函数包装成属性访问。你自己写描述符的场景通常是"重复的字段校验"(如上),与 C 里"每个字段手写 `setter`"对照,描述符一次写好、处处复用。

C(手写访问器)	Python(描述符)
每个字段一对 <code>getter/setter</code> ,复制粘贴	描述符写一次,挂在任意字段
校验逻辑散落在各字段函数里	校验集中在 <code>__set__</code> ,一处修改全局生效
新增字段忘了加访问器,数据裸奔	类属性上挂描述符,漏挂即属性不存在
访问器改名,调用方全部要改	实例属性语法不变,升级无感知

§10.5 元类:class 语句背后的构造过程

类也是对象,由**元类**创建——默认元类是 `type`。 `class Foo:` 在运行期等价于调用 `type("Foo", (object,), namespace)`。自定义元类继承 `type`,重写 `__new__` 拦截"类创建"这一刻,实现 AOP 式增强:

```

class TimedMeta(type):
    """元类:给所有方法自动包一层计时日志"""

    def __new__(mcls, name, bases, namespace):
        for key, value in list(namespace.items()):
            if callable(value) and not key.startswith("__"):
                namespace[key] = mcls._wrap(value)
        return super().__new__(mcls, name, bases, namespace)

    @staticmethod
    def _wrap(func):
        import time
        def wrapper(*args, **kwargs):
            start = time.perf_counter()
            result = func(*args, **kwargs)
            print(f"{func.__name__} 耗时 "
                  f"{{(time.perf_counter() - start) * 1000:.2f}} ms")
            return result
        return wrapper

class Worker(metaclass=TimedMeta):
    def job(self) -> None:
        for _ in range(1000000):
            pass

Worker().job()                                # 自动打印耗时,类体零改动

```

类创建三步:收集类体命名空间 → 调元类构造新类 → 绑定名字。自定义元类常用场景:框架层"注册所有子类"、"自动生成方法"、"统一接口校验"。代价是隐式魔法:新人看到 Worker 里没写计时,却输出了耗时——所以元类只该出现在库与框架层,业务代码慎用。

想判断一个自动化是否该用元类,问三个问题:收益是否覆盖所有使用者、失败模式是否可解释(报错信息是否指向问题根源)、新人能否在五分钟内看懂。三个都答是,才值得写。

§10.6 __init_subclass__:更亲民的子类钩子

Python 3.6+ 提供了元类场景 90% 的替代品: `__init_subclass__` 在"有人继承我"时被调用,直接写在普通类里,不需要元类。经典应用是插件注册表:

```

class Plugin:
    """插件基类:继承即注册,自动进入注册表"""
    registry: dict[str, type] = {}

    def __init_subclass__(cls, name: str = None, **kwargs) -> None:
        super().__init_subclass__(**kwargs)
        key = name or cls.__name__.lower()
        Plugin.registry[key] = cls    # 注册子类

class HttpChecker(Plugin, name="http"):    # 自定义注册名
    def run(self) -> str:
        return "检查 HTTP"

class DnsChecker(Plugin):
    def run(self) -> str:
        return "检查 DNS"

print(sorted(Plugin.registry))            # ['dns', 'http']
runner = Plugin.registry["http"]()
print(runner.run())                       # 检查 HTTP

```

插件系统、ORM 模型收集、策略注册——"继承即登记"让新增功能只写一个类,调度代码零改动。与 C 的注册表(手工登记函数指针数组)相比,Python 把登记动作自动化了。注意细节:关键字参数必须显式命名并处理(**kwargs 传给 super),否则子类自定义参数会报错。

C(手工注册)	Python(继承即注册)
新增插件要改注册表初始化函数	新增插件只写类,注册自动完成
漏登记直到运行期才暴露	注册发生在导入期,启动即报错
注册表与实现分离,易失同步	注册表与类定义同处一个模块,天然一致
函数指针数组的类型检查靠约定	检查器校验子类确实满足接口

误区:元类与 __init_subclass__ 混用踩坑。两者叠加时,执行顺序是"先元类构造类,再调 __init_subclass__";如果元类的 __new__ 里也做注册,会与 __init_subclass__ 重复执行或顺序颠倒。生产规则:同一份"子类收集"逻辑只选一个机制——优先 __init_subclass__(简单、可读),只有需要拦截"类创建过程本身"(改名、改基类、注入方法)才上元类。

§10.7 dataclass:数据类的元编程红利

dataclass 是标准库对"数据载体类"的元编程自动化:声明字段即自动生成 __init__、__repr__、__eq__,还能参与比较与不可变:

```

from dataclasses import dataclass, field

@dataclass(frozen=True)           # frozen:不可变,可哈希
class Product:
    sku: str
    name: str
    price: float = 0.0
    tags: list = field(default_factory=list) # 可变默认值必须 factory

    @property
    def total(self) -> float:
        return self.price

p = Product("P-001", "机械键盘", 399.0)
print(p)                          # Product(sku='P-001', ...)
print(p == Product("P-001", "机械键盘", 399.0)) # True
# p.price = 1                     # frozen=True 时报错

```

与 C 写"字段 + 初始化 + 打印 + 比较"的样板对照,dataclass 一次省掉几十行。三个要点:字段默认值直接写在标注后;可变默认值(list/dict)必须用 field(default_factory=) 否则所有实例共享同一对象(与第4章可变默认参数同源);frozen=True 让实例不可变从而可哈希,适合作为 dict 的 key。

dataclass 与字典的边界:临时打包几个值用 dict 够快,但"字段固定、需要方法、需要校验"的数据结构一律 dataclass——它把散落的键名拼接变成了拼写检查器能拦的字段访问,这是生产代码可维护性的关键差异。

§10.8 生产实践:元编程的边界

元编程是把双刃剑:自动化的另一面是隐式。三条生产纪律:① **只写"一次"与"多处收益"的自动化**——注册表、校验字段、统一日志值得;为省两行代码写元类不值。② 元编程代码必须有单元测试与文档,它的失败模式(属性不存在、注册遗漏)比普通代码难排查。③ 团队约定:业务模块禁用元类,框架层允许,__init_subclass__ 全项目可用。

工程建议 1:把 mypy 纳入 CI,标注覆盖率不用 100%,但新代码必须过检。Protocol + TypeVar + Generic 组合,让"运行期动态、检查期静态"成为项目常态——这是 Python 大型项目的唯一正解。

工程建议 2:数据类统一用 @dataclass,禁止手写 __init__/_repr__/_eq__ 样板;校验逻辑放 property 或 field 描述符,让非法数据在构造时失败——这比"运行到深处才爆"便宜一个数量级。

本章速查表

场景	写法
可空标注	<code>X None (3.10+) / Optional[X]</code>
联合类型	<code>A B (3.10+) / Union[A, B]</code>
内置泛型	<code>list[int] (3.9+); 3.8 用 List[int]</code>
泛型函数	<code>T = TypeVar("T"); def f(x: list[T]) -> T</code>
泛型类	<code>class Stack(Generic[T]):</code>
限制类型变量	<code>TypeVar("T", bound=Base)</code>
结构化接口	<code>class Payable(Protocol): def pay(self) -> float: ...</code>
运行期协议检查	<code>@runtime_checkable</code> + <code>isinstance</code> (仅方法存在性)
字段校验复用	自定义描述符(<code>__get__</code> / <code>__set__</code>)挂在类属性
类创建拦截	元类继承 <code>type</code> , 重写 <code>__new__</code>
继承即注册	<code>def __init_subclass__(cls, **kwargs) + registry</code>
数据类	<code>@dataclass(frozen=True)</code> + <code>field(default_factory=)</code>
前向引用	<code>from __future__ import annotations</code>
静态检查	<code>mypy src/</code> , 纳入 CI
元类纪律	仅框架层使用, 业务模块禁用
样板禁止	手写 <code>__init__</code> / <code>__repr__</code> 前先想 <code>dataclass</code>

最小必会清单

1. 能写出 3.10+ 与 3.8 两套兼容的标注(联合、可空、内置泛型)
2. 能用 `TypeVar` + `Generic` 写出泛型函数与泛型容器, 并解释 `T` 的追踪
3. 能用 `Protocol` 定义结构化接口, 并说明它与继承接口、鸭子类型的区别
4. 能解释描述符的 `__get__`/`__set__` 机制与存储模式, 并说明 `property` 的原理
5. 能用 `__init_subclass__` 实现"继承即注册", 并说清它与元类的分工
6. 能用 `@dataclass` 声明数据类, 并解释 `default_factory` 与 `frozen` 的用途

自测题

Q

Q1. 类型标注在运行期到底做什么?为什么说"标注只对检查器有意义"?

答案:运行期基本不做事:函数标注存进 `__annotations__` 属性供内省,但传参类型错误不会报错。校验由 `mypy` 等静态检查器在开发期完成。正因如此,标注写错不会崩,但检查器会立刻指出——所以标注的收益在"开发期拦截错误"与"文档化契约",不在运行期防御。

Q

Q2. `def first(items: list[T]) -> T` 中, `T` 与 `C` 的 `void*` 有什么区别?

答案:`void*` 在编译期丢失元素类型,取用要强转,转错运行期崩溃;`T` 由检查器按"同一位置类型一致"规则推导并全程追踪,`first(["a"])` 返回 `str`,`first([1])` 返回 `int`,传错类型在检查期就报错。运行期两者都不存在——Python 的 `T` 不生成任何代码。

Q

Q3. Protocol 与抽象基类(ABC)有什么区别?什么时候用哪个?

答案:ABC 是"继承式"契约:必须显式继承并实现抽象方法,是运行时强制;Protocol 是"结构化"契约:类型长得像即满足,无需继承,是检查期强制。选型:定义框架接口、需要运行期 `isinstance` 强制用 ABC;想让第三方类型零侵入地参与你的多态(鸭子类型静态化)用 Protocol。

Q

Q4. 描述符的 `__set__` 里写 `obj.xxx = value` 为什么递归?正确存值方式是什么?

答案:`obj.xxx` 触发属性查找,找到类上的描述符,再调 `__set__`,而 `__set__` 里又执行 `obj.xxx` 赋值.....无限递归直到 `RecursionError`。正确方式:把值存进 `obj.__dict__["xxx"]`,`__get__` 也读同一个 `key`;命名约定是描述符构造时接收存储名,与描述符自身的属性名一致。

Q

Q5. `__init_subclass__` 里为什么要 `super().__init_subclass__(**kwargs)`?不传会怎样?

答案:子类声明中的关键字参数(如 `name="http"`)先被 `class` 语句收集,传给 `__init_subclass__`; `super()` 调用把参数沿继承链继续传递,让父类的钩子也执行。若吞掉 `kwargs` 或漏调 `super`,深层继承时祖先钩子不执行,注册表缺类——这是"继承即注册"链断裂的头号原因。

Q

Q6. `@dataclass(frozen=True)` 的实例为什么能当 `dict` 的 `key`? `list` 字段为什么必须 `default_factory`?

答案:`frozen=True` 生成 `__setattr__` 拦截赋值,实例不可变, `dataclass` 自动生成基于字段的 `__hash__`,因此可哈希可做 `key`。`list` 是可变对象,若直接 `default=[]`,所有实例共享同一个列表(第4章可变默认参数坑的类级版本);`default_factory=list` 每次实例化调用一次工厂,生成独立列表。

章末实验题:类型安全的插件系统

实验 10•1 泛型检查器与插件注册表(覆盖:§10.2–§10.6)

需求:① 用 Protocol 定义 Checker 接口(方法 `check() -> Result`);② 用 `dataclass` 定义 `Result(code: int, message: str)`,支持比较;③ 用 `__init_subclass__` 实现"继承即注册"的插件注册表,支持自定义注册名;④ 实现至少两个插件(如 `HttpChecker`、`SqlChecker`),检查逻辑用正则与 `sleep` 模拟;⑤ 用 `TypeVar` + `Generic` 写一个泛型函数 `run_all(checkers: list[C]) -> list[Result]`,要求返回类型与传入类型对应;⑥ `main` 从注册表按名字取插件并全部运行,输出汇总;⑦ 跑一遍 `mypy` 确认零错误。

覆盖知识点:Protocol 结构化协议 (§10.3)/`TypeVar` 与 `Generic`(§10.2)/`__init_subclass__` 注册表 (§10.6)/`dataclass`(§10.7)。

实验环境:Python 3.10+,单文件 `plugins.py` 。

```

# plugins.py — 类型安全的插件注册表(覆盖第10章全部知识点)
import re
import time
from dataclasses import dataclass
from typing import Protocol, TypeVar

@dataclass(frozen=True)
class Result:
    """检查结果:不可变,可按 code 比较排序"""
    code: int # 0=通过,1=警告,2=失败
    message: str

    @property
    def ok(self) -> bool:
        return self.code == 0

class Checker(Protocol):
    """结构化协议:有 check() 即插件,无需继承"""

    def check(self) -> Result: ...

class Plugin:
    """插件基类:继承即自动注册,支持自定义注册名"""
    registry: dict[str, type] = {}

    def __init_subclass__(cls, name: str = None, **kwargs) -> None:
        super().__init_subclass__(**kwargs)
        key = name or cls.__name__.lower()
        if key in Plugin.registry:
            raise ValueError(f"插件名重复: {key}")
        Plugin.registry[key] = cls

class HttpChecker(Plugin, name="http"):
    """检查 HTTP 服务:模拟探测"""

    def __init__(self, url: str) -> None:
        self.url = url

    def check(self) -> Result:
        time.sleep(0.05) # 模拟网络探测
        if not re.match(r"^https?:/", self.url):
            return Result(2, f"URL 格式不合法: {self.url}")
        return Result(0, f"{self.url} 可达")

class SqlChecker(Plugin):

    def __init__(self, sql: str) -> None:
        self.sql = sql

    def check(self) -> Result:

```

```

    if re.search(r"drop\s+table|;\s*--", self.sql, re.I):
        return Result(2, "检测到危险 SQL 模式")
    if re.search(r"\bselect\b", self.sql, re.I):
        return Result(1, "SELECT 语句请走只读账号")
    return Result(0, "SQL 检查通过")

```

```

C = TypeVar("C", bound=Checker)

def run_all(checkers: list[C]) -> list[Result]:
    """泛型函数:接受任意 Checker 列表,返回对应结果"""
    return [c.check() for c in checkers]

def main() -> None:
    print("已注册插件:", sorted(Plugin.registry))
    http_cls = Plugin.registry["http"]          # 按名取类
    sql_cls = Plugin.registry["sqlchecker"]
    jobs = [http_cls("https://example.com"),    # 工厂式构造
            http_cls("ftp://bad.example.com"),
            sql_cls("select * from users; -- x"),
            sql_cls("update accounts set bal=0")]
    results = run_all(jobs)                      # 泛型调用
    for r in results:
        print(f"[{'通过' if r.ok else '失败'}] {r.message}")
    print(f"汇总: {sum(1 for r in results if r.ok)}/"
          f"{len(results)} 通过")

if __name__ == "__main__":
    main()

```

设计说明:① 注册名冲突在注册瞬间抛 `ValueError`,把"插件名重复"这类配置错误提前到导入期;② `Result` 用 `frozen dataclass`,不可变且自动带 `__eq__`/`__repr__`,正好呼应 §10.7;③ `run_all` 用 `TypeVar(bound=Checker)` 保证参数与返回的类型联动,乱传非 `Checker` 会被 `mypy` 拦截;④ `Plugin` 基类与业务插件分层,新增插件只写一个类——扩展系统不修改调度代码,这就是开闭原则的元编程实现;⑤ 最后在项目根执行 `python -m mypy plugins.py`,看到 `Success` 输出即类型安全达成。

下一章预告:第11章 反射与动态特性——`dir/inspect/type` 动态造类、`__getattr__` 与属性拦截、猴子补丁的双刃剑。让程序在运行期自我审视、自我改造。